

Creating accessible presentations with `ltx-talk`

Timothy Prescott

Abstract

We discuss how to create accessible PDF presentations using `ltx-talk`, which is similar to `beamer`. We give several examples, including how to adapt common Beamer styles. We also discuss necessary adaptations to satisfy a common accessibility checker available in several LMSs.

1 Introduction

In *TUGboat* 46:2 [1], Fischer and Mittelbach reported that

The `beamer` class is incompatible with the tagging code and due to the way it is implemented, it is not easily possible to change this. Joseph Wright therefore decided to write a new presentation class called `ltx-talk` with proper tagging support as a replacement for `beamer`.

Our purpose here is to explore this replacement.

2 The Basics

A `ltx-talk` presentation must begin with

```
\DocumentMetadata{}
\documentclass{ltx-talk}
```

But we will probably want a `\DocumentMetadata` more like

```
\DocumentMetadata{
  lang=en,
  tagging=on,
  pdfstandard=ua-2,
%  check-tagging=status,
  tagging-setup={
%    math/alt/use,
%    role/new-tag=frametitle/H1,
    math/setup=mathml-SE } }
```

Also, documents must be compiled using `lualatex` (preferable) or `pdflatex`.

The class documentation is available via `texdoc ltx-talk` on a command line, or at ctan.org/pkg/ltx-talk. This links to several examples at www.texdev.net/ltx-talk/examples/. If we want to customize the document, it is important to note that `ltx-talk` makes use of *templates*, so that we will want to familiarize ourselves with `texdoc lttemplates`

Broadly, we can view a `ltx-talk` as a properly tagged $\text{\LaTeX}$ document that incorporates presentation ideas such as overlays and the geometry. This means that “how to” questions often revolve around how something should be done in a $\text{\LaTeX}$ document

or a `beamer` presentation, but we will examine several examples that do not fit into those categories.

If we want to define a new overlay aware command, we should use the `xparse` syntax

```
\NewDocumentCommand{\mycommand}
{ s D<>{1-} ... }{ ... }
```

where the `s` indicates an optional star (if we want our command to look for that), the `D<>{all}` indicates the overlay specification with a default value *all* (or `d<>` if a default value doesn’t make sense), and then we have whatever other arguments we’d like our function to take.

For example, an upcoming

```
\NewDocumentCommand{\visiblealert}{d<> m}{%
  \visible<#1->{\alert<#1>{#2}}}
```

creates a new command `\visiblealert` that takes an overlay specification as `#1` and then a mandatory argument as `#2`. Its content will be visible on slides from `#1` onward, and its content will be alerted on the slide given by `#1`.

`ltx-talk` defines three colors: alert (red), example (green), and structure (blue). All three are slightly darker than standard 100%/0%/0% in RGB colors to enhance contrast for accessibility.

If a frame needs to use verbatim or listings to change the catcodes, then `ltx-talk` needs a `{frame*}` environment (this is similar to `beamer`’s concept of a *fragile* frame). As with most things related to verbatim, such a frame cannot appear as the argument of another command.

3 Overlays

`ltx-talk` modifies many common commands to accept an overlay specification (after an optional star, but before other arguments), so that the commands do nothing outside of the specified slides. `ltx-talk` also defines the usual overlay commands `\only`, `\uncover`, `\visible`, `\invisible`, and their environments `{\action}env`.

4 Sections

Within a frame, the sectioning commands `\section`, `\subsection`, and `\subsubsection` have no affect. (The other common sectioning commands should not appear in a presentation and are not defined.) Outside of the frames, the sectioning commands for sections with frames contribute to the table of contents, printed by the usual `\tableofcontents`. Before any sectioning command, this will present an outline of the presentation, using the usual `tocdepth` counter to determine how deep the levels should be displayed (the default is 2 to display up to `\subsection`). After a sectioning command, this will present the same

outline, but with the non-current sections mostly transparent.

By default, `ltx-talk` sets frame titles to be H4. The Ally accessibility checker, however, requires that the heading structure begin at H1 and descend by steps from there. We can specify that frametitles will be H1 by uncommenting the appropriate line of `\DocumentMetadata`.

5 Examples

5.1 Lists

Because `ltx-talk` has adapted the `beamer` overlay specification concept, we can still use

```
\begin{itemize}[<+>]
```

```
...
```

```
\end{itemize}
```

to create a list that we uncover item by item. (We generally prefer allowing the audience to take in the entire slide at their leisure instead of forcing their attention to bounce back and forth, but recognize that others are allowed to have their incorrect opinions on the matter.) If we also want to make use of the more powerful listing environments created by templates (see `texdoc blocks-doc`), then the `<+>` value goes with the key `action-spec`. For example,

```
\newlength\mylabelwidth
\setlength\mylabelwidth{\widthof{tagged PDFs }}
\begin{frame}
\frametitle{A Strangely Sporadic Glossary}
\begin{description}[
label-width=\mylabelwidth,
label-align=left,
left-margin=\mylabelwidth,
action-spec=<+>, ]
\item[\LaTeX] \TeX\ and one or two
additional macros
\item[\texttt{beamer}] presentation
class for \LaTeX
\item[tagged PDFs] Computers can read PDFs
\item[\texttt{ltx-talk}] tagged
\texttt{beamer} PDF
\end{description}
\end{frame}
```

creates four slides where each item is uncovered in turn, culminating in Figure 1.

For tagging purposes, and when creating a printed handout of the slides, repeating four slides with only slightly varying information doesn't make sense. Therefore, only the final slide is represented in those modes.

5.2 Matrix Multiplication

As another example, we were able to demonstrate matrix multiplication using

A Strangely Sporadic Glossary

$\LaTeX$	$\TeX$ and one or two additional macros
beamer	presentation class for $\LaTeX$
tagged PDFs	Computers can read PDFs
ltx-talk	tagged beamer PDF

Figure 1: Output of example `{description}` code.

```
\begin{frame}[tag-slides=1-]
\frametitle{Matrix Multiplication Example,
Step \arabic{slide}}
\NewDocumentCommand{\visablealert}{d<m}{%
\visible<#1->{\alert<#1>{#2}}}
\newcommand{\colvect}[1]{%
\begin{pmatrix}#1\end{pmatrix}}
\begin{gather*}
\begin{pmatrix}
\alert<2,3>{1}&\alert<2,3>{2}\\
\alert<4,5>{4}&\alert<4,5>{5}
\end{pmatrix}
\begin{pmatrix}
\alert<2,4>{-2}&\alert<3,5>{4}\\
\alert<2,4>{2}&\alert<3,5>{-5}
\end{pmatrix}
\visible<2-6>{
}=
\begin{pmatrix}
\visablealert<2>{2}&
\visablealert<3>{-6}\\
\visablealert<4>{2}&
\visablealert<5>{-9}
\end{pmatrix}
\\[\bigskipamount]
\vphantom{\colvect{1\\2}}%
\visible<2-5>{
\alert{
\only<2,3>{\colvect{1\\2}}
\only<4,5>{\colvect{4\\5}}
\bullet
\only<2,4>{\colvect{-2\\2}}
\only<3,5>{\colvect{4\\-5}}
=
\only<2>{2}\only<3>{-6}
\only<4>{2}\only<5>{-9}}
\end{gather*}
\end{frame}
```

The previous content works in `beamer` with the only necessary changes being to remove the `tag-slides=1-` and adjust the `\arabic{slide}`. This creates a frame of six slides, as seen in Figure 2.

Matrix Multiplication Example, Step 1	Matrix Multiplication Example, Step 2
$\begin{pmatrix} 1 & 2 \\ 4 & 5 \end{pmatrix} \begin{pmatrix} -2 & 4 \\ 2 & -5 \end{pmatrix}$	$\begin{pmatrix} 1 & 2 \\ 4 & 5 \end{pmatrix} \begin{pmatrix} -2 & 4 \\ 2 & -5 \end{pmatrix} = \begin{pmatrix} 2 & -6 \\ 2 & -9 \end{pmatrix}$ $\begin{pmatrix} 1 \\ 2 \end{pmatrix} \cdot \begin{pmatrix} -2 \\ 2 \end{pmatrix} = 2$
Matrix Multiplication Example, Step 3	Matrix Multiplication Example, Step 4
$\begin{pmatrix} 1 & 2 \\ 4 & 5 \end{pmatrix} \begin{pmatrix} -2 & 4 \\ 2 & -5 \end{pmatrix} = \begin{pmatrix} 2 & -6 \\ 2 & -9 \end{pmatrix}$ $\begin{pmatrix} 1 \\ 2 \end{pmatrix} \cdot \begin{pmatrix} 4 \\ -5 \end{pmatrix} = -6$	$\begin{pmatrix} 1 & 2 \\ 4 & 5 \end{pmatrix} \begin{pmatrix} -2 & 4 \\ 2 & -5 \end{pmatrix} = \begin{pmatrix} 2 & -6 \\ 2 & -9 \end{pmatrix}$ $\begin{pmatrix} 4 \\ 5 \end{pmatrix} \cdot \begin{pmatrix} -2 \\ 2 \end{pmatrix} = 2$
Matrix Multiplication Example, Step 5	Matrix Multiplication Example, Step 6
$\begin{pmatrix} 1 & 2 \\ 4 & 5 \end{pmatrix} \begin{pmatrix} -2 & 4 \\ 2 & -5 \end{pmatrix} = \begin{pmatrix} 2 & -6 \\ 2 & -9 \end{pmatrix}$ $\begin{pmatrix} 4 \\ 5 \end{pmatrix} \cdot \begin{pmatrix} 4 \\ -5 \end{pmatrix} = -9$	$\begin{pmatrix} 1 & 2 \\ 4 & 5 \end{pmatrix} \begin{pmatrix} -2 & 4 \\ 2 & -5 \end{pmatrix} = \begin{pmatrix} 2 & -6 \\ 2 & -9 \end{pmatrix}$

Figure 2: Output of example Matrix Multiplication code.

It is worthwhile to comment on the `tag-slides=1-` argument. As we remarked with the previous example, it is not useful to tag slides with slightly varying information. Consequently, `ltx-talk` would normally only tag the final slide in the frame. Pedagogically, we would prefer to have `ltx-talk tag all slides` in the frame, which is accomplished by the `1-` specification. As part of the overlay specifications, `tag-slides` also recognizes the variable `n` as “the last slide”, so that the default argument is `tag-slides=n`.

There is an additional complication if we want the `handout` mode of this example (with `beamer` as well). By default, the overlay specifications are *flattened*, so that anything that happens on one slide happens on the handout. This causes the `\alert` to apply to all of the items and the `\only` entries to all be visible.

There are two solutions to this problem. If we want the handout to follow the `projector` mode and its tagging, we can change all of the overlay specifications from `<spec>` to `<spec>|handout:<spec>`. This indicates that the `handout` specification will follow the default `projector` specification.

Alternatively, we can have only the last slide in the handout by using

6 Customizations

6.1 Adjusting the Header and Footer

A universal task in presentations is to tailor the header and footer. These are created in `ltx-talk` using templates, which allows convenient customization. (But we must first remark that these templates are still being developed; one should consult the doc-

Table 1: Default values of the header type `talk` template

Key	Default Value
background-color	(none)
color	structure
font	<code>\normalfont</code>
height	topmargin (10 mm) + headsep (2 mm)
left-hspace	leftmargin (10 mm)
right-hspace	rightmargin (10 mm)
print-frame-title	true

Table 2: Default values of the footer type `talk` template

Key	Default Value
background-color	(none)
color	black
element-order	(empty)
font	<code>\tiny</code>
left-hspace	leftmargin (10 mm)
right-hspace	rightmargin (10 mm)
separator	<code>\hfil</code>

umentation if code here does not work as expected.) The default values of the header template are listed in Table 1. We can adjust the values by using `\EditInstance{header}{std}{<key=value>}`. Note that the font key is intended for non title elements. Adjusting the font of the title is probably better done with `\EditInstance{frametitle}{header}{font=...}`.

My university encourages a presentation template that has a logo in the top right of the header. This cannot be done with the existing template, but can be accomplished with

```
\NewCommandCopy\oldframetitle\frametitle
\RenewDocumentCommand{\frametitle}{m}{%
  \oldframetitle{#1\hfill
    \includegraphics[height=2ex,artifact]
      {logfile}}}
```

The default values of the footer template are listed in Table 2. The `element-order` key specifies what information appears and is a comma separated list of footer-elements (see Table 3), such as `element-order={title,author,date,framenum}`. The `totalframes` footer-element displays as an integer. Outside of the footer, we can access that using `\RefProperty{lastpage}{totalframes}`.

6.2 Adjusting the Title Page

We could

```
\EditTemplateDefaults{titlepage}{talk}
  {<new key values>}
```

to adjust the title page, but that is only necessary if we plan to create multiple title pages. The default `\maketitle` is equivalent to

```
\maketitle[
  element-order      = {
    title, subtitle, author, institute, date},
  framestyle         = talk,
  horizontal-alignment = center,
  vertical-alignment  = center ]
```

The value for `framestyle` is passed to `\pagestyle`. In addition to the default and `plain` (no header or footer), we can also use `wallpaper` which removes the footer and the title (but not the background color of the header).

6.3 Creating Custom Elements

There are two steps to creating a custom element for a title page. The first is to declare its template instance:

```
\DeclareInstance{titlepage-element}{myelement}
{talk}{...}
```

where we use the final argument to change any default template options. `ltx-talk` will then use `\@myelement`, which we can define with the usual

```
\makeatletter
\newcommand\@myelement{...}
\makeatother
```

Similarly, to creating a custom footer element, we first declare its template instance:

```
\DeclareInstance{footer-element}{myelement}
{talk}{...}
```

`ltx-talk` will then use `\@shortmyelement` or `\@myelement` (favoring the `short` command if both exist). If we don't want to change the catcode of `@`, we can define the desired command with

```
\ExpandArgs{c}\newcommand{@myelement}{...}
```

6.4 More Customizations

The templates and instances created by `ltx-talk` are summarized in Table 3. The `floatenv` template adjusts the horizontal alignment of figures and tables using

```
\EditInstance{floatenv}{std}
{horizontal-alignment = <alignment>}
```

where `<alignment>` is one of `left` (default), `center`, or `right`. The `hidden` template adjusts the transparency of hidden items using

```
\EditInstance{hidden}{std}{opacity=<value>}
```

where the `<value>` ranges from 0 (default invisible) to 1 (fully visible). We may see information about these templates and instances during compilation and in the log file by using the template commands

Table 3: Instances and types in the template `talk` in `ltx-talk`

type	instances
titlepage	
titlepage-element	author, date, institute, title, subtitle
header	std, wallpaper
frametitle	header
footer	std, wallpaper
footer-element	date, author, title, subtitle, institute, framenummer, totalframes
floatenv	std
hidden	std

- `\ShowTemplateDefaults{<type>}{talk}`
- `\ShowTemplateInterface{<type>}{talk}`
- `\ShowTemplateVariables{<type>}{talk}`
- `\ShowTemplateCode{<type>}{talk}`
- `\ShowInstanceValues{<type>}{<instance>}`

7 Future Work

As `ltx-talk`'s reason for existence is proper tagging, please let us know of any tagging difficulties that you find.

We are intensely interested in feedback from screen reader users (as opposed to automated validators) as to what heading structure is useful to them. Such users are encouraged to get in contact with us.

References

- [1] U. Fischer, F. Mittelbach. News from the L^AT_EX Tagged PDF project: 2025. *TUGboat* 46(2):244–247, 2025. doi.org/10.47397/tb/46-2/tb143fischer-tagging25

◇ Timothy Prescott
University of North Dakota
101 Cornell Street Stop 8376
Grand Forks ND, 58203
USA
timothy dot prescott (at) UND
dot edu
ORCID 0009-0009-0555-8074